11 — Mathematics for Machine Learning (Deep Dive)

Time: ~5-6 hours | **Level**: Intermediate → Advanced

What you'll learn:

- Eigenvalues, eigenvectors, and their connection to PCA
- SVD: the Swiss army knife of linear algebra
- Optimization theory: loss surfaces, convexity, saddle points
- Gradient descent variants: SGD, Momentum, Adam from scratch
- Information theory: entropy, cross-entropy, KL divergence
- Bayesian thinking: priors, posteriors, MAP estimation
- Numerical stability: why your training can explode or vanish

Prerequisites: Notebook 01 (ML Foundations), basic calculus and linear algebra

---

Why Math Matters for ML Engineering

You don't need to prove theorems — but understanding *why* Adam converges faster than SGD, or *why* BatchNorm helps training, requires mathematical intuition. This notebook builds that intuition with code.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Eigenvalues & Eigenvectors → PCA Connection

Core idea: An eigenvector of a matrix A is a direction that A only *stretches* (not rotates).

$$A\mathbf{v} = \lambda\mathbf{v}$$

- $\mathbf{v}$ = eigenvector (direction)
- λ = eigenvalue (stretch factor)

Why it matters for ML: PCA finds the eigenvectors of the covariance matrix → the directions of maximum variance in your data.

In [ ]:
```
# ── Eigenvectors & PCA from scratch ──────────────────────────
from sklearn.datasets import load_iris
```

```
# Load data
X = load_iris().data[:, :2]  # 2D for visualization
X_centered = X - X.mean(axis=0)

# Covariance matrix
cov_matrix = np.cov(X_centered.T)
print("Covariance matrix:")
print(cov_matrix)

# Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
# Sort by largest eigenvalue
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print(f"\nEigenvalues: {eigenvalues}")
print(f"Variance explained: {eigenvalues / eigenvalues.sum() * 100}%")

# Plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Original data with eigenvectors
axes[0].scatter(X_centered[:, 0], X_centered[:, 1], alpha=0.5, s=30)
for i, (val, vec) in enumerate(zip(eigenvalues, eigenvectors.T)):
    axes[0].arrow(0, 0, vec[0]*val, vec[1]*val, head_width=0.05,
                  color=['red', 'blue'][i], linewidth=2, label=f'PC{i+1} (λ=
axes[0].set_title('Data with Principal Components')
axes[0].legend()
axes[0].set_aspect('equal')

# Projected data
X_projected = X_centered @ eigenvectors
axes[1].scatter(X_projected[:, 0], np.zeros_like(X_projected[:, 0]), alpha=0
axes[1].set_title('Data projected onto PC1 (1D)')
axes[1].set_xlabel('PC1')

plt.suptitle('PCA = Eigenvectors of Covariance Matrix', fontsize=14)
plt.tight_layout()
plt.show()
```

2. SVD — Singular Value Decomposition

$$A = U\Sigma V^T$$

- U = left singular vectors (column space)
- Σ = singular values (diagonal, importance)
- V^T = right singular vectors (row space)

Applications in ML:

- **Dimensionality reduction** (truncated SVD = LSA for text)

- **Image compression** (keep top-k singular values)
- **Recommendation systems** (matrix factorization)
- **Pseudoinverse** (solving overdetermined systems)

In [ ]:
```python
# ——— SVD for image compression ——————————————————————————

# Create a sample image (gradient + pattern)
x = np.linspace(0, 4*np.pi, 200)
y = np.linspace(0, 4*np.pi, 200)
X_grid, Y_grid = np.meshgrid(x, y)
image = np.sin(X_grid) * np.cos(Y_grid) + 0.5 * np.sin(2*X_grid)

# SVD
U, S, Vt = np.linalg.svd(image, full_matrices=False)

# Reconstruct with different numbers of components
fig, axes = plt.subplots(1, 5, figsize=(20, 4))
ranks = [1, 5, 20, 50, 200]

for ax, k in zip(axes, ranks):
    reconstructed = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    compression = k * (200 + 200 + 1) / (200 * 200) * 100
    ax.imshow(reconstructed, cmap='viridis')
    ax.set_title(f'Rank {k}\n({compression:.1f}% of original)')
    ax.axis('off')

plt.suptitle('SVD Image Compression: More singular values = more detail', fo
plt.tight_layout()
plt.show()

# Show singular value spectrum
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(S, 'o-', markersize=3)
ax.set_xlabel('Index')
ax.set_ylabel('Singular Value')
ax.set_title('Singular Value Spectrum (steep drop = compressible)')
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```

3. Optimization Theory — Understanding Loss Landscapes

Key concepts:

- **Convex**: One global minimum (linear regression, SVMs)
- **Non-convex**: Multiple local minima (neural networks)
- **Saddle points**: Gradient is zero but it's neither min nor max — the *real* problem in high dimensions

In [ ]:
```python
# ── Optimization landscape visualization ──────────────────────────

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Convex
x = np.linspace(-3, 3, 100)
axes[0].plot(x, x**2, 'b-', linewidth=2)
axes[0].plot(0, 0, 'r*', markersize=15)
axes[0].set_title('Convex: One global minimum')
axes[0].set_xlabel('Parameter')
axes[0].set_ylabel('Loss')

# Non-convex with local minima
y_nonconvex = np.sin(3*x) + 0.5*x**2
axes[1].plot(x, y_nonconvex, 'b-', linewidth=2)
local_mins = [-1.8, 0.5]
for lm in local_mins:
    idx = np.argmin(np.abs(x - lm))
    axes[1].plot(x[idx], y_nonconvex[idx], 'r*', markersize=15)
axes[1].set_title('Non-convex: Multiple local minima')
axes[1].set_xlabel('Parameter')

# Saddle point (2D)
from mpl_toolkits.mplot3d import Axes3D
ax3d = fig.add_subplot(133, projection='3d')
x_3d = np.linspace(-2, 2, 50)
y_3d = np.linspace(-2, 2, 50)
X3, Y3 = np.meshgrid(x_3d, y_3d)
Z3 = X3**2 - Y3**2  # Saddle
ax3d.plot_surface(X3, Y3, Z3, cmap='coolwarm', alpha=0.7)
ax3d.scatter([0], [0], [0], color='red', s=100, zorder=5)
ax3d.set_title('Saddle point: min in x, max in y')
axes[2].set_visible(False)

plt.suptitle('Optimization Landscapes in ML', fontsize=14)
plt.tight_layout()
plt.show()
```

4. Gradient Descent Variants — Implemented from Scratch

Optimizer	Update Rule	Key Idea
SGD	$\theta -= \alpha \cdot g$	Basic gradient step
Momentum	$v = \beta v + g;\ \theta -= \alpha v$	Accumulate velocity
RMSProp	Adaptive learning rate per parameter	Scale by running avg of g^2
Adam	Momentum + RMSProp + bias correction	Best of both worlds

In [ ]:
```python
# ── Gradient descent variants from scratch ────────────────────────
```

```python
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr
    def step(self, params, grads):
        return params - self.lr * grads

class Momentum:
    def __init__(self, lr=0.01, beta=0.9):
        self.lr, self.beta = lr, beta
        self.v = None
    def step(self, params, grads):
        if self.v is None:
            self.v = np.zeros_like(params)
        self.v = self.beta * self.v + grads
        return params - self.lr * self.v

class Adam:
    def __init__(self, lr=0.01, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr, self.beta1, self.beta2, self.eps = lr, beta1, beta2, eps
        self.m = self.v = None
        self.t = 0
    def step(self, params, grads):
        self.t += 1
        if self.m is None:
            self.m = np.zeros_like(params)
            self.v = np.zeros_like(params)
        self.m = self.beta1 * self.m + (1 - self.beta1) * grads
        self.v = self.beta2 * self.v + (1 - self.beta2) * grads**2
        m_hat = self.m / (1 - self.beta1**self.t)
        v_hat = self.v / (1 - self.beta2**self.t)
        return params - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

# Test on Rosenbrock function: f(x,y) = (1-x)^2 + 100(y-x^2)^2
def rosenbrock(p):
    return (1 - p[0])**2 + 100 * (p[1] - p[0]**2)**2

def rosenbrock_grad(p):
    dx = -2*(1-p[0]) - 400*p[0]*(p[1]-p[0]**2)
    dy = 200*(p[1]-p[0]**2)
    return np.array([dx, dy])

# Run each optimizer
optimizers = {'SGD': SGD(lr=0.001), 'Momentum': Momentum(lr=0.001), 'Adam':
histories = {}

for name, opt in optimizers.items():
    params = np.array([-1.0, 1.0])
    history = [params.copy()]
    for _ in range(500):
        grads = rosenbrock_grad(params)
        grads = np.clip(grads, -10, 10)  # Gradient clipping for stability
        params = opt.step(params, grads)
        history.append(params.copy())
    histories[name] = np.array(history)

# Plot convergence paths
```

```
fig, ax = plt.subplots(figsize=(10, 8))
x_plot = np.linspace(-1.5, 1.5, 200)
y_plot = np.linspace(-0.5, 2.0, 200)
X_plot, Y_plot = np.meshgrid(x_plot, y_plot)
Z_plot = (1-X_plot)**2 + 100*(Y_plot-X_plot**2)**2

ax.contour(X_plot, Y_plot, np.log10(Z_plot + 1), levels=30, cmap='gray', alp
colors = {'SGD': 'blue', 'Momentum': 'green', 'Adam': 'red'}
for name, hist in histories.items():
    ax.plot(hist[:200, 0], hist[:200, 1], '-', color=colors[name], label=nam
ax.plot(1, 1, 'r*', markersize=20, label='Optimum (1,1)')
ax.set_title('Optimizer Comparison on Rosenbrock Function')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend()
plt.tight_layout()
plt.show()
```

5. Information Theory — The Language of ML Losses

| Concept | Formula | ML Use | |---------|---------|--------| | **Entropy** | $H(p) = -\sum p_i \log p_i$ | Measures uncertainty | | **Cross-entropy** | $H(p, q) = -\sum p_i \log q_i$ | Classification loss | | **KL divergence** | $D_{KL}(p\|\|q) = \sum p_i \log \frac{p_i}{q_i}$ | VAE loss, knowledge distillation |

Key insight: Cross-entropy loss = Entropy + KL divergence

$$H(p, q) = H(p) + D_{KL}(p||q)$$

Minimizing cross-entropy loss → minimizing KL divergence from true distribution.

In [ ]:
```
# ——— Information theory: entropy, cross-entropy, KL divergence ———

def entropy(p):
    p = np.array(p)
    p = p[p > 0]  # Avoid log(0)
    return -np.sum(p * np.log2(p))

def cross_entropy(p, q):
    p, q = np.array(p), np.array(q)
    q = np.clip(q, 1e-10, 1.0)
    return -np.sum(p * np.log2(q))

def kl_divergence(p, q):
    p, q = np.array(p), np.array(q)
    q = np.clip(q, 1e-10, 1.0)
    mask = p > 0
    return np.sum(p[mask] * np.log2(p[mask] / q[mask]))

# Example: predicting cat vs dog
true_dist = [0.8, 0.2]  # 80% cat, 20% dog (true distribution)
```

```
predictions = {
    'Perfect': [0.8, 0.2],
    'Good': [0.7, 0.3],
    'Bad': [0.5, 0.5],
    'Wrong': [0.2, 0.8],
}

print(f"True distribution: {true_dist}")
print(f"Entropy H(true): {entropy(true_dist):.4f} bits\n")

print(f"{'Prediction':<12} {'CE Loss':>10} {'KL Div':>10}")
print("-" * 35)
for name, q in predictions.items():
    ce = cross_entropy(true_dist, q)
    kl = kl_divergence(true_dist, q)
    print(f"{name:<12} {ce:>10.4f} {kl:>10.4f}")

print(f"\nNote: CE = H(true) + KL = {entropy(true_dist):.4f} + KL")
print("Minimizing CE loss = minimizing KL divergence from truth")
```

6. Bayesian Thinking — Priors, Posteriors, and Regularization

Bayes' theorem:

$$P(\theta|D) = \frac{P(D|\theta) \cdot P(\theta)}{P(D)}$$

| Term | Name | ML Equivalent | |------|------|---------------| | $P(\theta)$ | Prior | Regularization (L2 = Gaussian prior) | | $P(D|\theta)$ | Likelihood | Model fit to data | | $P(\theta|D)$ | Posterior | Updated belief after seeing data |

Key insight: L2 regularization = assuming parameters come from a Gaussian prior centered at 0.

In [ ]:
```
# ── Bayesian update visualization ──────────────────────────────

from scipy import stats

x = np.linspace(-5, 10, 1000)

# Prior: we believe the parameter is near 0
prior = stats.norm(loc=0, scale=2)

# Likelihood: data suggests parameter is near 5
data_points = [4.5, 5.2, 4.8, 5.1, 4.9]  # Observations
likelihood_mean = np.mean(data_points)
likelihood_std = np.std(data_points) / np.sqrt(len(data_points))
likelihood = stats.norm(loc=likelihood_mean, scale=likelihood_std)

# Posterior (conjugate Gaussian: analytical solution)
prior_var = prior.var()
```

```
lik_var = likelihood.var()
post_var = 1 / (1/prior_var + 1/lik_var)
post_mean = post_var * (prior.mean()/prior_var + likelihood_mean/lik_var)
posterior = stats.norm(loc=post_mean, scale=np.sqrt(post_var))

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(x, prior.pdf(x), 'b--', linewidth=2, label=f'Prior: N(0, 2²)')
ax.plot(x, likelihood.pdf(x), 'g--', linewidth=2, label=f'Likelihood: N({lik
ax.fill_between(x, posterior.pdf(x), alpha=0.3, color='red')
ax.plot(x, posterior.pdf(x), 'r-', linewidth=2, label=f'Posterior: N({post_m
ax.set_xlabel('Parameter value')
ax.set_ylabel('Probability density')
ax.set_title('Bayesian Update: Prior × Likelihood → Posterior')
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print("The posterior is a COMPROMISE between prior and data:")
print(f"  Prior mean:      {prior.mean():.2f} (our initial belief)")
print(f"  Data mean:       {likelihood_mean:.2f} (what data says)")
print(f"  Posterior mean:  {post_mean:.2f} (compromise)")
print(f"\nStrong prior (small variance) → posterior closer to prior (= more
print(f"Lots of data (small likelihood variance) → posterior closer to data
```

7. Numerical Stability — Why Training Explodes or Vanishes

Common issues:

1. **Overflow**: softmax with large values → inf
2. **Underflow**: probabilities → 0 → log(0) → -inf
3. **Vanishing gradients**: deep networks with sigmoid → gradients → 0
4. **Exploding gradients**: RNNs → gradients → inf

Solutions:

- Log-sum-exp trick for numerically stable softmax
- Gradient clipping
- Careful initialization (Xavier, Kaiming)
- Architecture choices (ReLU over sigmoid, residual connections)

In [ ]:
```
# ─── Numerical stability: the log-sum-exp trick ──────────────────

# UNSTABLE softmax
def softmax_unstable(x):
    exp_x = np.exp(x)  # Can overflow!
    return exp_x / exp_x.sum()

# STABLE softmax (subtract max)
def softmax_stable(x):
```

```
    x_shifted = x - np.max(x)  # Prevents overflow
    exp_x = np.exp(x_shifted)
    return exp_x / exp_x.sum()

# Demonstrate the problem
print("Softmax with large values:")
x_small = np.array([1.0, 2.0, 3.0])
x_large = np.array([1000.0, 2000.0, 3000.0])

print(f"  Small inputs {x_small}: stable={softmax_stable(x_small)}")

try:
    result = softmax_unstable(x_large)
    print(f"  Large inputs (unstable): {result}")
except Exception as e:
    print(f"  Large inputs (unstable): OVERFLOW!")

import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    unstable_result = softmax_unstable(x_large)
    print(f"  Large inputs (unstable): {unstable_result} (NaN!)")

print(f"  Large inputs (stable):   {softmax_stable(x_large)}")

# Gradient clipping
print("\nGradient clipping:")
gradient = np.array([100.0, -200.0, 50.0])
max_norm = 10.0
grad_norm = np.linalg.norm(gradient)
if grad_norm > max_norm:
    gradient_clipped = gradient * max_norm / grad_norm
    print(f"  Original gradient norm: {grad_norm:.1f}")
    print(f"  Clipped gradient norm:  {np.linalg.norm(gradient_clipped):.1f}
    print(f"  Direction preserved: {np.allclose(gradient / grad_norm, gradie
```

Key Takeaways

Concept	One-Line Summary
Eigenvalues/PCA	PCA finds eigenvectors of covariance matrix = directions of max variance
SVD	Universal matrix decomposition: compression, denoising, recommendations
Optimization	Neural nets are non-convex; saddle points matter more than local minima
Adam	Momentum + adaptive learning rates + bias correction = the default choice
Cross-entropy	CE loss = entropy + KL divergence; minimizing CE = minimizing distance to truth

Concept	One-Line Summary
Bayesian thinking	L2 regularization = Gaussian prior; more data → less regularization effect
Numerical stability	Always use log-sum-exp trick, gradient clipping, proper initialization

What to study next:

- **Notebook 12**: Advanced Classical ML (apply these concepts to real problems)
- **Notebook 13**: CNNs and training techniques (where optimization theory meets practice)